

Chapter 72

Python & pandas Question Bank

Answer the Python and pandas questions that come up across screening calls, live-coding rounds, and take-homes for every data role · recognize the classic Python gotchas (mutable defaults, is vs ==, late-binding closures) before they bite you live · write idiomatic, correctly-vectorized pandas...

Approved chapter · v1

Contents

Chapter 72. Python & pandas Question Bank

72.1 Python fundamentals: the gotchas that catch experienced candidates

72.2 Predict the output: basic to advanced tricky questions

72.3 Functions, generators, and decorators

72.4 pandas fundamentals: Series, DataFrames, and selection

72.5 pandas: grouping, joining, and reshaping

72.6 pandas: performance, missing data, strings, and dates

72.7 Debugging and live-coding walk-throughs

Common mistakes and how to spot them

In the real world: the report that "randomly" gave different numbers each run

Tools

The project

Final-week revision list

Key terms

Where this leads

Chapter 72. Python & pandas Question Bank

Part VIII — The Interview Playbook

You will learn to: answer the Python and pandas questions that come up across screening calls, live-coding rounds, and take-homes for every data role · recognize the classic Python gotchas (mutable defaults, `is` vs `==`, late-binding closures) before they bite you live · write idiomatic, correctly-vectorized pandas instead of slow, easy-to-get-wrong loops · debug broken pandas code the way a live round actually tests you.

How this chapter is built. Same format as Chapters 70–73: every core question leads with a "Remember it as..." hook, a one-line answer, a compact tier table. Rapid-fire sections are scan tables. **Every code snippet in this chapter was actually executed** in this exact Python and pandas version, not recalled from memory. Section order was planned before writing: language fundamentals and their gotchas first, then functions and iteration, then pandas from basic selection through to debugging and live-coding, each section building on the last.

A note on currency. This chapter is written against **pandas 3.0.2**, where two genuinely recent, significant changes affect classic interview answers. First, **Copy-on-Write is always enabled** and can no longer even be turned off, which retires the once-common "explain `SettingWithCopyWarning`" question (§72.4). Second, **string columns now default to a dedicated `str` dtype**, not the older, more generic `object` dtype every pre-3.0 pandas tutorial describes (Q72-019, Q72-035). Both the old behavior (which you may still meet on a team running an older pandas) and the current one are covered, clearly labeled, rather than only teaching what's now out of date.

Learn it in pointers are at chapter level (this chat doesn't have this book's Python-fundamentals and pandas chapters' approved text to check exact section numbers against: flagged for a re-check once available, the same convention used elsewhere in this part).

72.1 Python fundamentals: the gotchas that catch experienced candidates

Q72-001 · Why does calling a function with a mutable default argument twice give a surprising result?

Remember it as: A default argument is created once, when the function is defined, not fresh on every call. A mutable default remembers everything you've ever done to it.

Answer in one line: A default argument value is evaluated exactly once, at function definition time, and reused across every call, so a mutable default (like a list) accumulates changes across calls instead of starting fresh each time.

Verified, live:

```
def add_item(item, basket=[]):
    basket.append(item)
    return basket

add_item("crate")    # ['crate']
add_item("lid")      # ['crate', 'lid'] -- not just ['lid']!
```

```
['crate', 'lid'] ['crate', 'lid'] -- both calls return the SAME list object
```

Tier	What to say
Passes	"You shouldn't use mutable defaults" (correct instinct, no demonstration of why)
Strong	The mechanism above: the empty list is created once at definition time and silently shared across every call that doesn't pass its own <code>basket</code>
Extra points	+ [Validate] the live proof above shows both return values are literally <code>True</code> for <code>is</code> , the exact same object in memory, not two separate lists that happen to look alike + [Business] the fix: <code>basket=None</code> , then <code>if basket is None: basket = []</code> inside the function body, creating a fresh list every call

Likely follow-ups: Does this happen with immutable defaults like `basket=0`? (No: *immutable defaults can't be mutated in place, so there's nothing to accumulate.*) Where else in Python does "created once, reused" show up? **Red flag:** not knowing why this specific bug happens, or claiming it's random/inconsistent behavior. **Learn it in:** Chapter 72's own fundamentals section (this book's Python-basics chapter; exact number to confirm).

Q72-002 · What's the difference between `is` and `==`, and when does mixing them up cause a real bug?

Remember it as: `==` asks "do these look the same?" `is` asks "are these literally the same object in memory?"

Answer in one line: `==` compares values for equality (and can be customized per type); `is` compares object identity, whether two names point to the exact same object in memory.

Verified, live:

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
```

```
a == b: True    (same values, different objects)
a is b: False   (different objects in memory)
a is c: True    (c literally points to the same object as a)
```

Tier	What to say
Passes	" <code>is</code> checks identity, <code>==</code> checks equality" (correct, no example of the bug it causes)
Strong	The three-way comparison above, and a concrete case where confusing them bites: <code>if my_list is []:</code> is always <code>False</code> , even for an empty list, because it's asking "is this the exact same empty-list object I have in mind," not "is this list empty"

Tier	What to say
Extra points	+ [Edge cases] small integers and short strings are cached by CPython, so <code>a = 5; b = 5; a is b</code> can return <code>True</code> purely as an implementation detail, not a language guarantee: never rely on <code>is</code> for value comparison on anything but <code>None</code> , <code>True</code> , and <code>False</code>

Likely follow-ups: Why is `if x is None:` the recommended style over `if x == None:`? What's `__eq__` and how does customizing it change `==`'s behavior for your own classes? **Red flag:** using `is` to compare values (numbers, strings, lists) rather than reserving it for identity checks like `is None`.
Learn it in: Chapter 72's own fundamentals section.

Q72-003 · What's the difference between a shallow copy and a deep copy, demonstrated on nested data?

Remember it as: A shallow copy duplicates the outer box. A deep copy duplicates everything inside it too, all the way down.

Answer in one line: `.copy()` (or `copy.copy()`) creates a new outer container but the inner objects it holds are still shared with the original; `copy.deepcopy()` recursively copies everything, so nothing is shared at any level.

Verified, live:

```
import copy
orig = {"items": [1, 2, 3]}
shallow = orig.copy()
deep = copy.deepcopy(orig)
shallow["items"].append(4)
```

```
orig["items"]:    [1, 2, 3, 4]    -- changed! shallow copy shared the inner list
shallow["items"]: [1, 2, 3, 4]
deep["items"]:    [1, 2, 3]      -- unaffected, deep copy is fully independent
```

Tier	What to say
Passes	"Deep copy copies everything, shallow copy doesn't" (correct, no demonstration)
Strong	The nested-dictionary example above, showing the <i>original</i> data unexpectedly mutating through a shallow copy of something you thought was safely duplicated
Extra points	+ [Validate] the live proof: mutating <code>shallow["items"]</code> silently changed <code>orig["items"]</code> too, exactly the kind of bug that's easy to miss in a larger codebase + [Business] this exact trap shows up constantly with nested config dictionaries or lists of records passed between functions: always ask "does this container hold other mutable containers" before reaching for a plain <code>.copy()</code>

Likely follow-ups: Does this matter for a flat dictionary with no nested containers? (No: *shallow copy is completely safe when nothing inside is itself mutable and shared.*) What's the performance cost of `deepcopy` on a large nested structure? **Red flag:** assuming `.copy()` always produces a fully independent object. **Learn it in:** Chapter 72's own fundamentals section.

Q72-004 · Why does this loop of lambdas return the same value for every function, and how do you fix it?

Remember it as: A closure remembers the variable, not its value at the time the closure was made. By the time you call it, the loop has already finished.

Answer in one line: A lambda created inside a loop captures the variable `i` by reference, not its value at that point in the loop, so by the time any of the lambdas are actually called, the loop has finished and every one of them sees `i`'s final value.

Verified, live:

```
funcs = [lambda: i for i in range(3)]
[f() for f in funcs]
```

```
[2, 2, 2] -- every function returns 2, the loop's final value, not 0, 1, 2 as intuition suggests
```

Fixed, by capturing the value as a default argument, evaluated immediately at definition time:

```
funcs_fixed = [lambda i=i: i for i in range(3)]
[f() for f in funcs_fixed]
```

```
[0, 1, 2]
```

Tier	What to say
Passes	"Something about closures" (senses it's a known gotcha, can't explain it or fix it)
Strong	The reference-vs-value explanation above, and the <code>lambda i=i: i</code> fix, exploiting the fact that default arguments (per Q72-001) are evaluated once, immediately, which is exactly the behavior needed here
Extra points	+ [Validate] both real outputs above: the broken <code>[2, 2, 2]</code> and the fixed <code>[0, 1, 2]</code> , same loop structure, one line changed + [Depth] this is the same "evaluated once vs. evaluated every time" distinction from Q72-001, just showing up as a bug instead of a feature there

Likely follow-ups: Does this happen with a regular `for` loop and `def`, not just `lambda`? (Yes, the same closure-over-variable behavior applies to any nested function.) What's a cleaner fix than the default-argument trick? (A factory function, or `functools.partial`.) **Red flag:** not recognizing this as a known, named Python behavior (late-binding closures), guessing at random causes. **Learn it in:** Chapter 72's own fundamentals section.

Rapid-fire, 72.1

#	Question	One-line answer	Extra point
Q72-005	What's the difference between a list and a tuple?	Lists are mutable, tuples are immutable; tuples can be dictionary keys and set members, lists can't	[Business] using a tuple signals "this shouldn't change," which is documentation as much as a technical constraint

#	Question	One-line answer	Extra point
Q72-006	What does Python's None represent, and how do you correctly check for it?	The absence of a value; check with <code>is None</code> , never <code>== None</code>	[Edge cases] <code>== None</code> usually works too, but <code>is None</code> is the correct idiom since it can't be fooled by a custom <code>__eq__</code> that overrides equality
Q72-007	What's the difference between <code>*args</code> and <code>**kwargs</code> ?	<code>*args</code> collects extra positional arguments into a tuple; <code>**kwargs</code> collects extra keyword arguments into a dict	[Depth] verified: <code>describe(1,2,3, name="crate", qty=5) → args=(1, 2, 3), kwargs={'name': 'crate', 'qty': 5}</code>
Q72-008	What's a list comprehension, and is it always faster than a loop?	<code>[expr for item in iterable if condition]</code> , usually faster than an equivalent explicit loop because the iteration happens in optimized C code internally	[Validate] verified equal correctness: <code>[x*2 for x in data if x%2==0]</code> produces exactly the same result as the explicit loop version, line for line
Q72-009	What does <code>__init__</code> do, and is it a constructor?	It initializes a newly created object's attributes; the actual object creation happens in <code>__new__</code> , which <code>__init__</code> runs after	[Edge cases] almost nobody overrides <code>__new__</code> in ordinary data work; knowing it exists is usually enough unless the role is specifically about building frameworks

72.2 Predict the output: basic to advanced tricky questions

The purest interview format: a short snippet, no explanation, just "what does this print, and why." Ordered easy to hard. Every single one below was actually run; the printed output is real, not a remembered answer.

Q72-036 · `a = int("20", 4); print(a)` : what prints, and why?

Remember it as: The second argument to `int()` isn't a data type: it's the base the string should be read in. "20" in base 4 means 2 fours plus 0 ones.

Answer in one line: `int(string, base)` parses `string` as a number written in the given base, not base 10, so "20" in base 4 means $2 \times 4^1 + 0 \times 4^0 = 8$.

Verified, live:

```
a = int("20", 4)
print(a)
```

8

Tier	What to say
Passes	Guesses 20 (reading the string as if base didn't matter), or guesses randomly
Strong	Correctly computes 8, explaining the positional-value logic: digit "2" is in the 4s place, digit "0" is in the 1s place, $2 \times 4 + 0 \times 1 = 8$

Tier	What to say
Extra points	+ [Edge cases] <code>int("0x1A", 16)</code> and <code>int("0x1A", 0)</code> both correctly give 26: base 0 tells Python to <i>infer</i> the base from a prefix like <code>0x</code> , <code>0o</code> , or <code>0b</code> in the string itself, rather than requiring you to already know it + [Business] this shows up for real whenever a system hands you IDs, hashes, or flags encoded in hex or binary as plain text and you need to parse them back into actual integers

Likely follow-ups: What does `int("20")` give with no second argument at all? (20, base 10 is the default.) What happens if the string has a digit invalid for the given base, like `int("29", 4)`? (A `ValueError`, since 9 isn't a valid base-4 digit.) **Red flag:** assuming the second argument changes the output's type or format rather than how the input string is interpreted. **Learn it in:** Chapter 72's own fundamentals section.

Q72-037 • `print(0.1 + 0.2 == 0.3)` : what prints, and why?

Remember it as: Computers store most decimals as an approximation in binary, the same way $1/3$ has no exact end in decimal. 0.1 and 0.2 already aren't exactly what you typed, before you even add them.

Answer in one line: `False` : floating-point numbers are stored in binary, and most decimal fractions (0.1, 0.2, 0.3 included) have no exact binary representation, so tiny rounding errors accumulate and `0.1 + 0.2` lands a hair away from the binary value of `0.3`.

Verified, live:

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
```

```
0.30000000000000004
False
```

Tier	What to say
Passes	"Floating point is imprecise" (true, no mechanism, can't predict the actual behavior)
Strong	The binary-representation explanation above, and knows the correct fix is comparing within a small tolerance (<code>abs(a - b) < 1e-9</code>) or using the <code>decimal</code> module for money-like values, not <code>==</code> directly
Extra points	+ [Validate] the real printed value above, <code>0.30000000000000004</code> , not just "it's not exactly 0.3" as a vague claim + [Business] this is exactly why financial calculations (Chapter 12/70's revenue sums) are safer in <code>decimal.Decimal</code> or fixed-point integer cents than raw <code>float</code> , and why comparing two computed revenue totals with <code>==</code> in a test suite is a latent bug waiting to happen

Likely follow-ups: How would you safely compare two floats for "close enough" equality? Does this affect `int` arithmetic the same way? (No: *integers in Python have no such precision limit.*) **Red flag:** claiming this is a Python-specific bug rather than how virtually all languages represent floating-point numbers (IEEE 754). **Learn it in:** Chapter 72's own fundamentals section.

Q72-038 • `print(2 ** 3 ** 2)` : what prints, and why?

Remember it as: Most operators go left to right. The power operator is the odd one out: it goes right to left.

Answer in one line: `512 : **` is right-associative, so this evaluates as `2 ** (3 ** 2) = 2 ** 9 = 512`, not `(2 ** 3) ** 2 = 8 ** 2 = 64`.

Verified, live:

```
print(2 ** 3 ** 2)
print(-2 ** 2)
```

```
512
-4
```

Tier	What to say
Passes	Guesses 64, assuming left-to-right like most other operators
Strong	Correctly computes 512, explicitly naming right-associativity as the reason
Extra points	+ [Edge cases] <code>-2 ** 2</code> gives <code>-4</code> , not <code>4</code> , because unary minus binds <i>looser</i> than <code>**</code> , so it's actually <code>-(2 ** 2)</code> , not <code>(-2) ** 2</code> : a second, related surprise in the same family of operator-precedence traps + [Validate] both real outputs above, together, since they're easy to get backwards from each other in memory

Likely follow-ups: What's the full operator precedence order for arithmetic in Python? How would you force left-to-right evaluation if you needed `(2**3)**2` instead? **Red flag:** confidently stating `**` is left-associative like `+` or `*`. **Learn it in:** Chapter 72's own fundamentals section.

Q72-039 · `print(0 or 5)` and `print(3 and 5)`: what prints, and why?

Remember it as: `and` / `or` in Python don't return `True/False`. They return whichever actual operand decided the answer.

Answer in one line: `or` returns the first operand if it's truthy, otherwise the second, regardless of type; `and` returns the first operand if it's falsy, otherwise the second: neither one converts its result to an actual boolean.

Verified, live:

```
print(0 or 5)          # 5    -- 0 is falsy, so `or` moves on to and returns 5
print(3 and 5)         # 5    -- 3 is truthy, so `and` moves on to and returns 5
print([] or 'default') # 'default'
print(None and 'x')    # None
```

```
5
5
default
None
```

Tier	What to say
Passes	"It returns <code>True</code> or <code>False</code> " (the single most common wrong assumption)
Strong	Correctly predicts all four real outputs above, explaining <code>or</code> / <code>and</code> return an actual operand, not a boolean

Tier	What to say
Extra points	+ [Business] this is exactly the mechanism behind the common Python idiom <code>value = user_input or "default"</code> , using <code>or</code> 's real behavior deliberately to provide a fallback, rather than a bug to avoid + [Edge cases] an empty list, empty string, <code>0</code> , and <code>None</code> are all falsy and behave identically here, which can surprise someone expecting only <code>None / False</code> to count

Likely follow-ups: What's the output of `print(bool(0 or 5))` instead? What are *all* the falsy values in Python? **Red flag:** assuming `and / or` always produce `True / False`. **Learn it in:** Chapter 72's own fundamentals section.

Q72-040 • `grid = [[0]*3]*3; grid[0][0] = 1; print(grid)` : what prints, and why?

Remember it as: `[[0]*3]*3` makes one inner list, then three names pointing at that same one list. Changing "one row" changes all of them at once.

Answer in one line: `[[1, 0, 0], [1, 0, 0], [1, 0, 0]]`: the outer `* 3` doesn't create three independent inner lists, it creates three references to the *exact same* inner list object, so mutating one row through any one of those references changes what looks like "every row" at once.

Verified, live:

```
grid = [[0]*3]*3
grid[0][0] = 1
print(grid)                                     # every row changed, not just the first
```

```
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

The correct way to build a genuinely independent grid:

```
grid2 = [[0]*3 for _ in range(3)]
grid2[0][0] = 1
print(grid2)
```

```
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

Tier	What to say
Passes	Predicts <code>[[1, 0, 0], [0, 0, 0], [0, 0, 0]]</code> , the intuitive-but-wrong answer for the broken version
Strong	Correctly predicts all three rows changing, and explains it's the exact same underlying bug family as Q72-003's shallow copy trap: <code>*</code> on a list of lists duplicates references, not the referenced objects
Extra points	+ [Validate] both real outputs above, the trap and its fix, side by side + [Business] this is a real, common bug in anyone's first attempt at initializing a 2D grid, matrix, or game board in Python, and it's exactly the kind of thing worth testing with a two-line check before trusting a larger program built on top of it

Likely follow-ups: Does the same trap apply to `[0, 0, 0] * 3`? (No: those *inner elements* are *immutable integers*, so there's nothing to accidentally share; the trap is specific to lists of mutable objects.) How would you build a 2D NumPy array instead, and does it have the same issue? **Red flag:**

not recognizing this as the same underlying "shared reference" mechanism as the mutable-default-argument and shallow-copy traps already covered. **Learn it in:** Chapter 72's own fundamentals section.

Rapid-fire, 72.2A: more predict-the-output, easy to hard

#	Snippet	Output	Why
Q72-041	<code>print(1 < 2 < 3)</code>	True	Python chains comparisons: this is <code>(1 < 2)</code> and <code>(2 < 3)</code> , both true, evaluated without recomputing <code>2</code> twice
Q72-042	<code>print('ab' * 3)</code>	ababab	String <code>*</code> repeats the string, the same repetition idea as list <code>*</code> , just on characters instead of elements
Q72-043	<code>print(sum([]))</code> and <code>print(sum([], 10))</code>	0 and 10	<code>sum()</code> 's second argument is the starting value, defaulting to 0: useful for summing a possibly-empty list without a separate empty check
Q72-044	<code>first, *rest = "hello"; print(first, rest)</code>	h ['e', 'l', 'l', 'o']	Unpacking works on any iterable, including strings (iterating character by character), not just lists and tuples
Q72-045	<code>c = int("1000"); d = int("1000"); print(c is d)</code>	False	1000 is outside CPython's small-integer cache (roughly -5 to 256); built at runtime via <code>int()</code> (so the compiler can't silently fold the two literals into one shared constant the way it sometimes does with identical literals typed directly in the same code), the two 1000s are genuinely separate objects, honestly demonstrating the cache's real limit
Q72-046	<code>class C: items = []</code> then <code>c1 = C(); c1.items.append(1); c2 = C(); print(c2.items)</code>	[1]	A mutable class attribute (not inside <code>__init__</code>) is shared by every instance of the class, the class-level cousin of Q72-001's mutable-default-argument bug
Q72-047	<code>print([x for x in range(5) if (y := x * 2) > 4])</code>	[3, 4]	The walrus operator <code>:=</code> assigns <i>and</i> returns a value in one expression, here inside a comprehension's filter condition; the printed list is the <code>x</code> values themselves (not <code>y</code>) that satisfy the condition

Rapid-fire, 72.2B: sequences specifically, going deeper

Same quick format, narrowed to one theme: lists, tuples, strings, and the operations that reveal how they actually differ under the hood.

#	Snippet	Output	Why
Q72-048	<code>a=[1,2]; b=a; a+= [3]; print(a, b, a is b)</code>	[1, 2, 3] [1, 2, 3] True	<code>+=</code> on a list calls <code>.extend()</code> internally, mutating the <i>same</i> object in place; <code>b</code> sees the change because <code>b</code> and <code>a</code> are two names for that one object
Q72-049	<code>c=[1,2]; d=c; c=c+[3]; print(c, d, c is d)</code>	[1, 2, 3] [1, 2] False	Plain <code>+</code> (no <code>=</code>) always builds a <i>new</i> list; reassigning <code>c</code> to it leaves <code>d</code> pointing at the original, untouched list: the direct contrast with Q72-048
Q72-050	<code>t=([1,2], 3); t[0].append(99); print(t)</code>	([1, 2, 99], 3)	A tuple is immutable about <i>what it points to</i> , not about what those things can do: you can't replace <code>t[0]</code> itself, but the list <code>t[0]</code> refers to can still be mutated freely

#	Snippet	Output	Why
Q72-051	<code>lst=[0,1,2,3,4,5]; print(lst[5:1:-1])</code>	<code>[5, 4, 3, 2]</code>	A negative step walks backward from index 5 down to (but not including) index 1; the stop bound is still exclusive, just approached from the other direction
Q72-052	<code>print([1,2,3] + (4,5))</code>	<code>TypeError: can only concatenate list (not "tuple") to list</code>	<code>+</code> between sequences requires matching types; unlike <code>*</code> (which works the same way on any sequence type), mixed-type <code>+</code> is a hard error, not silent coercion
Q72-053	<code>print(list(zip([1,2,3], [4,5])))</code>	<code>[(1, 4), (2, 5)]</code>	<code>zip()</code> silently stops at the <i>shorter</i> input with no warning or error: a common, quiet source of lost data when two "matching" lists turn out not to be the same length
Q72-054	<code>print(sorted([(1,'b'),(1,'a'), (0,'z')], key=lambda x: x[0]))</code>	<code>[(0, 'z'), (1, 'b'), (1, 'a')]</code>	Python's sort is stable: items that tie on the sort key keep their original relative order, so <code>(1,'b')</code> stays before <code>(1,'a')</code> exactly because it came first in the input

72.3 Functions, generators, and decorators

Q72-010 · What's the difference between a function that returns a list and one that's a generator, and why does it matter for a large dataset?

Remember it as: A list builds and holds everything in memory at once. A generator hands you one value at a time and forgets it immediately after.

Answer in one line: A function using `return [...]` (or a list comprehension) computes and stores every value in memory immediately; a function using `yield` produces values one at a time, on demand, using a small, constant amount of memory regardless of how many values there are in total.

Verified, live, generating one million squared numbers each way:

```
def squares_list(n): return [i*i for i in range(n)]
def squares_gen(n):
    for i in range(n):
        yield i*i
```

```
list size in memory:      8,448,728 bytes    (~8.4 MB, for 1,000,000 items)
generator size in memory:    200 bytes      (constant, regardless of how many items it will
eventually produce)
```

Tier	What to say
Passes	"Generators are more memory efficient" (true, no sense of the actual magnitude)
Strong	The mechanism above, plus correctly noting a generator can only be iterated once, unlike a list which can be looped over repeatedly

Tier	What to say
Extra points	+ [Validate] the real, dramatic numbers above: 8.4 MB versus 200 bytes, over 40,000 times smaller, for the exact same logical sequence of a million values + [Business] this is precisely why reading a huge file or database result set with a generator-based approach (or pandas' <code>chunksize</code> parameter, §72.7) avoids running out of memory on data that would never fit as one in-memory list or DataFrame

Likely follow-ups: When would you not want a generator, even for a large sequence? What's a generator expression, versus a generator function? **Red flag:** not knowing a generator can only be consumed once, or claiming it has no downsides at all. **Learn it in:** Chapter 72's own fundamentals section.

Q72-011 · Write a decorator that times how long a function takes to run

Remember it as: A decorator is a function that takes a function and returns a new function that wraps it: extra behavior added around the original, without changing the original's own code.

Answer in one line: `@decorator` above a function definition is shorthand for `function = decorator(function)`, and a typical decorator defines an inner `wrapper` function that does something extra, then calls the original function inside it.

Verified, live:

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
        t1 = time.perf_counter()
        print(f"{func.__name__} took {t1 - t0:.6f}s")
        return result
    return wrapper

@timer
def slow_sum(n):
    return sum(range(n))

slow_sum(1_000_000)
```

```
slow_sum took 0.012510s
result: 499999500000
```

Tier	What to say
Passes	Can describe what a decorator does in English, struggles to write one from scratch
Strong	The working <code>timer</code> decorator above, correctly using <code>*args</code> , <code>**kwargs</code> (Q72-007) so it works on any function's signature, not just one specific one
Extra points	+ [Validate] the real timing output and the correct underlying result (499,999,500,000, the actual sum of 0 to 999,999) shown together, proving the decorator adds behavior without changing the original function's return value + [Business] this exact pattern (a timing or logging decorator) is one of the most common real uses of decorators in production data code, wrapping pipeline steps to automatically log how long each one took

Likely follow-ups: What does `functools.wraps` do, and why is it usually added to a decorator like this? What if the decorated function needs to accept arguments itself, does this pattern already handle that? (Yes, via `*args`, `**kwargs` inside `wrapper`.) **Red flag:** a decorator that doesn't use `*args`, `**kwargs`, and so only works on functions with one specific, hard-coded signature. **Learn it in:** Chapter 72's own fundamentals section.

Rapid-fire, 72.2

#	Question	One-line answer	Extra point
Q72-012	What's a closure?	A function that "remembers" variables from the scope it was defined in, even after that outer scope has finished running	[Depth] Q72-004's late-binding bug is a closure behaving exactly as designed; the surprise is in <i>when</i> the variable's value is read, not that it's remembered at all
Q72-013	What does <code>functools.wraps</code> do inside a decorator?	Preserves the original function's name, docstring, and metadata on the wrapped version, which a plain decorator would otherwise overwrite with the wrapper's own	[Business] without it, debugging tools and documentation generators see the wrapper's generic name instead of the real function's, making stack traces confusing
Q72-014	What's the difference between a function and a method?	A method is a function defined inside a class, implicitly receiving the instance (<code>self</code>) as its first argument	[Edge cases] a <code>@staticmethod</code> is the exception, a function living inside a class that does <i>not</i> receive <code>self</code> at all
Q72-015	What does <code>*</code> alone (with nothing after it) do in a function signature?	Forces every argument after it to be passed by keyword only, not positionally	[Business] used in well-designed APIs to prevent a caller from accidentally passing arguments in the wrong order when several parameters share a similar type

72.4 pandas fundamentals: Series, DataFrames, and selection

Q72-016 · What's the difference between a pandas Series and a DataFrame, in one sentence?

Remember it as: A *Series* is one labeled column. A *DataFrame* is a table of them, sharing a common row index.

Answer in one line: A **Series** is a one-dimensional labeled array (essentially one column, with an index); a **DataFrame** is a two-dimensional table, a collection of Series sharing the same row index, each with its own name and data type.

Tier	What to say
Passes	"A DataFrame is like a table, a Series is like a column"
Strong	+ explicitly: <code>df["column"]</code> returns a Series; selecting multiple columns (<code>df[["a", "b"]]</code>) returns a DataFrame, even if it's only one column wide, because it's still a table structure

Tier	What to say
Extra points	+ [Edge cases] <code>df["column"]</code> (single brackets) returns a Series; <code>df[["column"]]</code> (double brackets) returns a one-column DataFrame: a very common source of a "why doesn't this method exist" error, since Series and DataFrame methods overlap but aren't identical

Likely follow-ups: How would you convert a Series back into a one-column DataFrame? What does a DataFrame's `.index` actually represent? **Red flag:** not knowing the single-bracket vs. double-bracket distinction, or why it matters. **Learn it in:** Chapter 72's own pandas section.

Q72-017 · Does `SettingWithCopyWarning` still happen in current pandas, and what changed?

Remember it as: *Pandas 3.0 made the old trap structurally impossible, not just quieter. Chained assignment now just works, safely, every time.*

Answer in one line: In pandas versions before 3.0, filtering a DataFrame (`sub = df[condition]`) and then assigning to a column on `sub` could silently fail to update the original `df`, or ambiguously might, triggering `SettingWithCopyWarning`; **pandas 3.0 enables Copy-on-Write permanently** (it can no longer even be disabled), which makes every such filtered subset a genuine, fully independent copy, so the old ambiguity, and the warning itself, no longer applies.

Verified, live, on pandas 3.0.2:

```
df = pd.DataFrame({"customer": ["A", "B", "C"], "category": ["Storage", "Kitchen", "Storage"],
                  "quantity": [3, 10, 2]})
sub = df[df["category"] == "Storage"]
sub["quantity"] = sub["quantity"] * 2    # no warning at all in pandas 3.0+
```

sub after assignment:				original df, completely unchanged:			
	customer	category	quantity		customer	category	quantity
0	A	Storage	6	0	A	Storage	3
2	C	Storage	4	1	B	Kitchen	10
				2	C	Storage	2

Tier	What to say
Passes	Describes the old <code>SettingWithCopyWarning</code> trap as if it's still current behavior, without checking the pandas version in use
Strong	Correctly distinguishes: on an older pandas (pre-3.0, still common in many production codebases as of this writing), the warning is real and the safe fix is <code>.copy()</code> before assigning; on pandas 3.0+, Copy-on-Write makes this automatically safe with no warning needed
Extra points	+ [Validate] the real pandas 3.0.2 output above: <code>sub</code> is correctly modified, and <code>df</code> is provably untouched, with zero warning raised + [Business] always confirm which pandas version an interviewer or team is actually running before answering confidently either way: this is exactly the kind of "fast-changing fact" that needs checking, not reciting from memory or older training

Likely follow-ups: What is Copy-on-Write, conceptually, in one sentence? What's the safe pattern to use on an older pandas version, if you don't control the environment? **Red flag:** confidently describing `SettingWithCopyWarning` as current behavior without qualifying which pandas version that applies to. **Learn it in:** Chapter 72's own pandas section.

Rapid-fire, 72.3

#	Question	One-line answer	Extra point
Q72-018	<code>.loc</code> vs. <code>.iloc</code> ?	Label-based selection (<code>.loc[row_label, col_label]</code>) / integer-position-based selection (<code>.iloc[row_number, col_number]</code>)	[Edge cases] <code>.loc</code> 's slicing is <i>inclusive</i> of the end label (<code>df.loc[0:2]</code> includes row 2); <code>.iloc</code> 's slicing follows normal Python rules and excludes the end (<code>df.iloc[0:2]</code> excludes row 2)
Q72-019	What does <code>df.dtypes</code> tell you, and why check it early?	The data type pandas inferred for each column	[Business] a numeric-looking column silently typed as text (<code>str</code> in pandas 3.0+, the older <code>object</code> dtype in earlier versions) usually means hidden non-numeric values mixed in, worth catching before any calculation runs on it
Q72-020	How do you check for and count duplicate rows?	<code>df.duplicated().sum()</code> counts them; <code>df[df.duplicated()]</code> shows them; <code>df.drop_duplicates()</code> removes them	[Edge cases] <code>.duplicated()</code> defaults to keeping the <i>first</i> occurrence as not-a-duplicate; <code>keep="last"</code> or <code>keep=False</code> change that behavior
Q72-021	What's <code>df.info()</code> useful for that <code>df.head()</code> isn't?	Row count, column data types, and non-null counts per column, all at once: a fast structural health check	[Business] the pandas equivalent of Chapter 71's SQL table-profiling query (Q71-047D): one call, several useful facts about data you've never seen

72.5 pandas: grouping, joining, and reshaping

Q72-022 · Group Riverstone-style order data by customer, computing total revenue, order count, and average quantity in one call

Remember it as: `.agg()` with named outputs beats calling `.sum()`, `.count()`, and `.mean()` separately: one pass over the data, self-documenting column names.

Answer in one line: `.groupby(col).agg(name=("column", "function"), ...)` computes several different aggregations, on potentially different columns, in a single grouped pass, with clean, self-chosen output column names.

Verified, live:

```
orders["revenue"] = orders["quantity"] * orders["unit_price"]
orders.groupby("customer").agg(
    total_revenue=("revenue", "sum"),
    orders=("order_id", "count"),
    avg_qty=("quantity", "mean"),
)
```

	total_revenue	orders	avg_qty
customer			
Coastal Foods	4900.0	1	5.0
Metro Mart	1370.0	2	6.0
Sharma Hardware	1960.0	3	3.0

Tier	What to say
Passes	Three separate <code>.groupby().sum()</code> , <code>.groupby().count()</code> , <code>.groupby().mean()</code> calls, then manually joining the results together: works, three passes over the data instead of one, more code to keep in sync
Strong	The single <code>.agg()</code> call above, with named outputs
Extra points	+ [Validate] the real output above reconciles: Sharma Hardware's 3 orders average 3.0 units each (their quantities were 3, 4, and 2), and 1960 total revenue matches summing their three order values by hand + [Business] named aggregation (<code>total_revenue=("revenue", "sum")</code>) produces immediately readable column names, versus the older <code>.agg({ "revenue": "sum" })</code> style, which leaves you renaming columns afterward anyway

Likely follow-ups: How would you group by two columns at once? What's the difference between `.agg()` and `.transform()`? **Red flag:** running multiple separate `.groupby()` calls and manually merging the results, instead of one `.agg()` call. **Learn it in:** Chapter 72's own pandas section.

Q72-023 · Merge an orders table with a customers table, and find any customer with zero orders

Remember it as: An inner join only shows what matches on both sides. Finding "nothing matched" needs an outer join, checked for the resulting blanks: the exact pandas version of Chapter 71's SQL LEFT JOIN / IS NULL pattern.

Answer in one line: `orders.merge(customers, on="customer", how="right")`, then filter for rows where a column that only exists in `orders` is `NaN`, mirroring exactly the SQL LEFT JOIN ... WHERE ... IS NULL pattern from Chapter 71 (Q71-001), just with `orders` and `customers` swapped so the `customers` table is the "keep everything" side.

Verified, live:

```
merged_right = orders.merge(customers, on="customer", how="right")
merged_right[merged_right["order_id"].isna()][["customer", "city"]]
```

```
   customer  city
6  Home Plus  Delhi
```

Tier	What to say
Passes	<code>orders.merge(customers, on="customer")</code> with the default <code>how="inner"</code> : silently drops customers with no orders entirely, rather than surfacing them
Strong	The <code>how="right"</code> version above, correctly finding the one customer (Home Plus) with no matching orders
Extra points	+ [Validate] the real output above, and a check that the default inner merge's row count equals the original <code>orders</code> row count exactly (<code>6 == 6</code>), confirming every order matched a known customer with nothing silently dropped on that side + [Depth] this is the exact same fan-out risk from Chapter 71's SQL joins: a duplicate <code>customer</code> name in the <code>customers</code> table would silently multiply matching order rows here too, pandas' <code>merge</code> follows the identical join-cardinality rules as SQL

Likely follow-ups: How would you do this with `how="outer"` instead, and what changes? What's `indicator=True` useful for on a merge like this? **Red flag:** using the default inner merge and never checking whether any expected rows silently disappeared. **Learn it in:** Chapter 72's own pandas

section (and Chapter 71 for the identical SQL-side logic).

Rapid-fire, 72.4

#	Question	One-line answer	Extra point
Q72-024	<code>pivot_table</code> vs. <code>groupby</code> ?	<code>pivot_table</code> reshapes grouped results into a wide table (categories become columns) / <code>groupby</code> keeps results in long/tall form	[Validate] verified: <code>orders.pivot_table(index="customer", columns="category", values="revenue", aggfunc="sum", fill_value=0)</code> correctly places each customer's spend under the right category column, zeros elsewhere
Q72-025	What does <code>fill_value=0</code> do in <code>pivot_table</code> , and why not leave it as the default?	Replaces cells with no matching data (a customer-category combination that never occurred) with 0 instead of <code>NaN</code>	[Edge cases] leaving <code>NaN</code> in a pivot meant for summing or charting can silently break downstream arithmetic; 0 is usually the semantically correct "this combination didn't happen"
Q72-026	What's <code>pd.concat()</code> for, and how is it different from <code>merge</code> ?	Stacks DataFrames together, by rows (default) or columns, based on position/index, not by matching key values	[Depth] <code>concat</code> is pandas' equivalent of SQL's <code>UNION ALL</code> (Chapter 71, Q71-006); <code>merge</code> is pandas' equivalent of a SQL <code>JOIN</code>
Q72-027	What does <code>.transform()</code> do that <code>.agg()</code> doesn't?	Returns a result the same length as the original DataFrame (one value per original row, broadcast back to its group), rather than one row per group	[Business] useful for adding a "this row's value as a % of its group's total" column directly onto the original, un-collapsed data

72.6 pandas: performance, missing data, strings, and dates

Q72-028 · Why is `.apply()` with `axis=1` almost always slower than a vectorized operation, and how much slower in practice?

Remember it as: `.apply(axis=1)` is a disguised Python for-loop. Vectorized pandas operations run in fast, compiled C code underneath, all at once.

Answer in one line: `.apply(func, axis=1)` calls `func` once per row, in plain Python, exactly as slow as an explicit loop; a vectorized operation (`df["a"] * df["b"]`) is computed all at once, underneath, in optimized compiled code, without ever looping in Python at all.

Verified, live, on 200,000 rows:

```
big.apply(lambda row: row["a"] * row["b"], axis=1) # row-by-row
big["a"] * big["b"]                               # vectorized
```

```

apply time:      0.9511 s
vectorized time: 0.000852 s
speedup:        1,116x
results match:   True

```

Tier	What to say
Passes	"Vectorized code is faster" (true, no sense of <i>how much</i> faster, no explanation)
Strong	The compiled-C-vs-Python-loop explanation above, correctly identifying <code>.apply(axis=1)</code> as a loop wearing a pandas costume
Extra points	+ [Validate] the real, dramatic measured number above: over a thousand times faster, for identical, verified-matching results, on 200,000 rows + [Business] on a genuinely large dataset, this exact mistake is the difference between a report that runs in under a second and one that times out or takes minutes, with zero change to the actual logic being computed

Likely follow-ups: Is `.apply()` ever the right choice? (Yes, *when the per-row logic genuinely can't be vectorized, complex conditional branching or calling an external function per row.*) What's `.apply()` on a single column (`axis=0`), and is it faster than the row-wise version? **Red flag:** defaulting to `.apply(axis=1)` for any row-wise arithmetic that could be expressed as a direct column operation instead. **Learn it in:** Chapter 72's own pandas section.

Q72-029 · Handle missing data in a numeric column: detect it, then decide between filling and dropping

Remember it as: Detect first, decide second. Filling and dropping are both valid, but only after you know how much is missing and why.

Answer in one line: `.isna().sum()` counts missing values per column; `.fillna(value)` replaces them (a mean, median, zero, or a forward-fill, depending on context); `.dropna(subset=[...])` removes rows missing a required value entirely, and the right choice depends on *why* the data is missing, not a default habit.

Verified, live:

```

orders2["unit_price"] = orders2["unit_price"] # (one value set to NaN for this demo)
orders2["unit_price"].isna().sum()
orders2["unit_price"].fillna(orders2["unit_price"].mean())
orders2.dropna(subset=["unit_price"]).shape

```

```

missing count: 1
fillna with mean: [320.0, 85.0, 393.0, 980.0, 260.0, 320.0] -- row 3 filled with the column's
mean (393.0)
dropna shape: (5, 7) -- one row removed entirely

```

Tier	What to say
Passes	"Use <code>.fillna()</code> or <code>.dropna()</code> " with no discussion of when each is appropriate
Strong	+ explicitly: filling with a mean/median is reasonable when a value is plausibly missing at random and you need every row for a later calculation; dropping is more honest when a missing value signals something you don't want to guess at, and losing a few rows doesn't bias the analysis

Tier	What to say
Extra points	+ [Business] filling silently, with no comment or downstream flag, can hide a real data-quality problem from anyone reading the resulting numbers later: pair a fill with a comment or a boolean "was_imputed" column when the choice matters + [Edge cases] <code>.fillna()</code> 's mean is computed <i>after</i> any earlier filtering; filling before you've decided your final row set can bake in a slightly wrong mean

Likely follow-ups: When would forward-fill (`.ffill()`) be more appropriate than a mean? How would you handle missing data differently for a categorical column versus a numeric one? **Red flag:** filling all missing values with 0 by default, regardless of what the column represents. **Learn it in:** Chapter 72's own pandas section.

Rapid-fire, 72.5

#	Question	One-line answer	Extra point
Q72-030	Extract the first word of each customer name using <code>.str</code> ?	<code>df["customer"].str.split().str[0]</code>	[Validate] verified: "Sharma Hardware" → "Sharma", "Metro Mart" → "Metro"
Q72-031	Get the day of the week from a date column?	<code>df["date"].dt.day_name()</code> , or <code>.dt.dayofweek</code> for a 0–6 integer	[Validate] verified: 2025-01-05 → "Sunday", 2025-01-06 → "Monday"
Q72-032	What does <code>df["col"].str.contains("text")</code> do with a missing value in the column?	Returns <code>NaN</code> for that row by default, not <code>False</code> , which can cause an unexpected error if the result is later used directly as a boolean filter	[Edge cases] pass <code>na=False</code> explicitly to treat missing values as non-matches instead of propagating <code>NaN</code> into a filter
Q72-033	How would you process a file too large to fit in memory, using pandas?	<code>pd.read_csv(path, chunksize=100_000)</code> returns an iterator of DataFrame chunks, processed one at a time instead of all at once	[Depth] this is the pandas-specific version of Q72-010's generator idea: bounded memory use regardless of the total file size

72.7 Debugging and live-coding walk-throughs

Q72-034 • Fix this function: it's supposed to return each customer's share of total revenue, but it always returns 0 or 1

Remember it as: Integer division in the wrong order truncates to zero before it ever gets the chance to be a meaningful fraction.

```
# BROKEN
def revenue_share(customer_revenue, total_revenue):
    return customer_revenue // total_revenue
```

What's wrong: `//` is integer (floor) division, not regular division; any `customer_revenue` smaller than `total_revenue` floors to 0, and any share close to or above 1 floors to 1 or higher: it can never return an actual fraction like 0.35.

```
# FIXED
def revenue_share(customer_revenue, total_revenue):
    return customer_revenue / total_revenue
```

Tier	What to say
Diagnosis	<code>//</code> truncates instead of dividing normally
Fix	A single-character change, <code>//</code> to <code>/</code>
Extra points	+ [Validate] test with real numbers: <code>revenue_share(1960, 4900)</code> gives <code>0</code> with the broken version and the correct <code>0.4</code> with the fix + [Edge cases] worth checking <code>total_revenue == 0</code> separately either way, since both <code>/</code> and <code>//</code> raise <code>ZeroDivisionError</code> on a zero denominator, and a customer aggregation that somehow produces a zero total is worth investigating on its own

Likely follow-ups: What's the difference between `/` and `//` in Python 3 specifically, versus Python 2's default behavior? How would this same bug look in pandas, applied to a whole column? **Red flag:** not immediately recognizing `//` as integer division, or assuming the bug is somewhere more complicated than the operator itself. **Learn it in:** Chapter 72's own fundamentals section.

Q72-035 · Walk-through: clean and validate a messy, realistic order export

What they're really testing: whether you have a repeatable process for a genuinely messy real file, not just a single trick.

Talked through live: "I'll check shape and dtypes first, look for duplicates, check for missing values column by column, and only then start fixing anything, since fixing before I understand what's actually wrong risks masking a bigger problem."

Verified, live, a deliberately messy small order export:

```
messy = pd.DataFrame({
    "order_id": [1001, 1002, 1002, 1004],
    "customer": [" Sharma Hardware", "Metro Mart", "Metro Mart", "Coastal Foods"],
    "quantity": ["3", "10", "10", None],
})
print(messy.duplicated(subset=["order_id"]).sum())      # 1 -- order 1002 appears twice
print(messy["quantity"].dtype)                         # str -- quantity is stored as text, not
numeric
print(messy["customer"].str.startswith(" ").sum())     # 1 -- a leading space that would break
an exact-match join
```

```
duplicate order_ids: 1
quantity dtype: str
customer names with a leading space: 1
```

The fix, applied in order:

```
clean = messy.drop_duplicates(subset=["order_id"])
clean["customer"] = clean["customer"].str.strip()
clean["quantity"] = pd.to_numeric(clean["quantity"], errors="coerce")
```

Extra-points moves demonstrated: **[Structure]** diagnosed before fixing, in a defined order. **[Validate]** every claim (a duplicate, a wrong dtype, a leading space) backed by an actual check, not assumed. **[Edge cases]** used `errors="coerce"` on the numeric conversion rather than letting a genuinely bad value crash the whole pipeline, turning it into a `NaN` that can be caught and reported instead.

Likely follow-ups: How would you decide whether to keep the first or last of a duplicate `order_id`? What would you do with the `NaN` quantity that `pd.to_numeric` might produce, if the original value genuinely wasn't a number? **Red flag:** starting to "fix" a file (dropping rows, filling values) before running any diagnostic checks on what's actually wrong with it. **Learn it in:** Chapter 72's own pandas section (and Chapter 71's SQL table-profiling walk-through, Q71-047D, for the identical discipline in SQL).

Common mistakes and how to spot them

Mistake	Symptom	Fix
A mutable default argument (<code>def f(x, items=[])</code>)	Calls silently accumulate state across unrelated calls	Use <code>items=None</code> , then <code>if items is None: items = []</code> inside
Using <code>is</code> to compare values	<code>if x is []:</code> never matches, even for an empty list	Reserve <code>is</code> for <code>None / True / False</code> ; use <code>==</code> for values
Assuming <code>.copy()</code> fully isolates nested data	Mutating a "copy" silently changes the original too	Use <code>copy.deepcopy()</code> when the container holds other mutable containers
Citing <code>SettingWithCopyWarning</code> as current pandas behavior with no version check	Confidently wrong on pandas 3.0+, where Copy-on-Write is always on	State which pandas version the answer applies to
<code>.apply(axis=1)</code> for simple row-wise arithmetic	1,000x+ slower than the vectorized equivalent, verified live	Use direct column operations (<code>df["a"] * df["b"]</code>) whenever possible
Inner-joining without checking for silently dropped rows	Missing customers/records go unnoticed	Compare merged row count against expectations; check with <code>how="right" / "outer"</code> when rows shouldn't disappear
Filling every missing value with 0 by habit	Real "unknown" values get treated as real zeros in later math	Decide fill vs. drop based on <i>why</i> the value is missing
"Fixing" a messy file before diagnosing it	Silently over-correcting, or missing the actual root problem	Check shape, dtypes, duplicates, and missing values first, in that order

In the real world: the report that "randomly" gave different numbers each run

Karan is asked to debug a weekly revenue report that occasionally produces a slightly different total for the same underlying data, depending on which order the input files happen to load in. He traces it to a helper function with a mutable default argument, exactly Q72-001's bug, used to accumulate a running list of "already counted" order IDs across calls, meant to reset for every new report run but never actually resetting, since the default list was created once when the module was first imported.

The fix was the one-line change Q72-001 already teaches (`orders_seen=None` instead of `orders_seen=[]`), but the part that mattered in the interview retelling wasn't the fix, it was how Karan found it: rather than guessing, he added a single `print(len(orders_seen))` at the top of the function and ran the report twice in the same Python session, watching the count grow between runs when it should have reset to zero each time. That one line of evidence turned a vague "it's inconsistent somehow" into an exact, provable diagnosis in under two minutes.

The lesson the hiring manager wrote in her notes afterward: *"Didn't guess. Added one line to make the invisible state visible, then the bug explained itself."* That's Chapter 69's Move 7 (validate), applied to debugging instead of a finished answer, and it's exactly the difference this chapter's format tries to model in every single question: don't just know the fix, show the evidence.

Tools

Python 3.12 and **pandas** 3.0.2, the exact versions every snippet in this chapter was run against.

`%timeit` in a Jupyter notebook (or the plain `time.perf_counter()` pattern used throughout this chapter) for any performance claim you make in an interview, rather than asserting a speedup from memory. **pandas' own release notes** are worth skimming before any interview at a company you know uses pandas heavily, since behavior (like Copy-on-Write, §72.4) does genuinely change between major versions.

The project

Goal: find and fix a real instance of at least three of this chapter's core bugs, in your own code or a public repository.

1. Search your own past projects for a mutable default argument (`def f(x, y=[])` or `y={}`) and fix it, the way Q72-001 does.
2. Take one place in your own code using `.apply(axis=1)` and rewrite it as a vectorized operation; measure the real speedup, the way Q72-028 did, don't estimate it.
3. Pick a merge or join in your own work and verify the row count is what you expect, catching any silently dropped or duplicated rows.

4. Run `.isna().sum()` on a real dataset you have access to, and for each column with missing data, decide and justify fill vs. drop, in one sentence each.
5. Write one decorator of your own (logging, timing, or retry logic) and apply it to a real function you use regularly.

Final-week revision list

Q72-001, Q72-002, Q72-003, Q72-004, Q72-010, Q72-011, Q72-016, Q72-017, Q72-022, Q72-023, Q72-028, Q72-029, Q72-034, Q72-035.

Key terms

mutable default argument · `is` vs. `==` · object identity · shallow copy · deep copy · closure · late binding · `*args` / `**kwargs` · list comprehension · generator · decorator · `functools.wraps` · positional-only / keyword-only arguments · pandas Series · pandas DataFrame · `.loc` vs. `.iloc` · Copy-on-Write · `SettingWithCopyWarning` (legacy) · `groupby().agg()` · named aggregation · `merge` (join types) · `pivot_table` · `concat` · `.transform()` vs. `.agg()` · vectorization · `.apply(axis=1)` · `.isna()` / `.fillna()` / `.dropna()` · `.str` accessor · `.dt` accessor · `chunksize` (large-file reading)

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 71, SQL Question Bank**, already covered the identical join-cardinality and NULL-handling logic this chapter's pandas `merge` questions (§72.5) reuse directly.
- **Chapter 72A, Data Structures & Algorithms Question Bank**, covers the classic DSA problems (arrays, trees, graphs) this chapter deliberately leaves aside in favor of pandas-specific, data-role-shaped coding.
- **Chapter 74, Machine Learning Question Bank**, assumes this chapter's pandas fluency as a starting point, not something to re-teach.